

We've entered Software 3.0, where LLMs are programmable computers that process prompts instead of code, fundamentally changing what it means to build software and requiring us to rethink everything from development workflows to infrastructure design.

[

The December Shift: From Helpful to Autonomous

- December 2024 marked a clear inflection point where agentic coding tools stopped requiring corrections and started producing consistently correct code chunks
- This wasn't incremental improvement—it was a paradigm shift from 'helpful assistant' to 'trusted executor' that many missed if they only used ChatGPT-style interfaces
- The transition enabled 'vibe coding': describing intent and trusting the system to implement it correctly without constant supervision
- Side projects exploded as the friction of implementation dropped dramatically, making previously tedious ideas suddenly tractable

I can't remember the last time I corrected it. I just trusted the system more and more, and then I was vibe coding.

Verifiability: The New Automation Frontier

- LLMs excel in domains where outputs can be verified (math, code) because frontier labs train them with reinforcement learning using verification rewards
- This creates 'jagged' capabilities: models can refactor 100K line codebases or find zero-days, yet suggest walking to a car wash 50 meters away to wash your car
- Capabilities peak where labs focus training effort. Chess improved dramatically from GPT-3.5 to GPT-4 not from general intelligence gains, but because chess data was added to pre-training
- Founders should identify verifiable domains not yet targeted by labs—these are tractable for fine-tuning and RL even without frontier-scale resources

How is it possible that state-of-the-art Opus 4.7 will simultaneously refactor a 100,000 line codebase yet tells me to walk to this car wash? This is insane.

What Humans Still Own

- Agents are powerful interns: excellent recall, fast execution, but weak on taste, judgment, and high-level design decisions
- Humans remain responsible for specs, aesthetics, and architectural coherence. Example: Menu Gen agent tried matching users by email address across Stripe and Google—technically functional but architecturally broken
- You no longer need to remember API details (keepdims vs keepdim, reshape vs permute), but you must understand fundamentals (tensor views, memory efficiency, storage vs computation tradeoffs)
- Models struggle with simplification and elegance—they produce bloated, copy-paste heavy code that works but lacks craft. This isn't fundamental; labs just haven't prioritized it in RL yet

You can outsource your thinking, but you can't outsource your understanding.

Animals vs Ghosts: Understanding LLM Nature

- LLMs aren't animal intelligences shaped by evolution, curiosity, or intrinsic motivation. They're 'ghosts' — statistical simulations shaped by data and reward functions
- Yelling at an LLM doesn't change its behavior. It has no emotional state, no persistent goals, no self-preservation instinct. Understanding this prevents anthropomorphization errors
- Pre-training provides statistical foundations; RL bolts on specialized capabilities, creating jagged performance profiles across domains
- This framing matters for deployment: treat them as powerful but alien tools, not as reasoning agents with human-like understanding or motivation

],

[

Software 3.0: Neural Networks as Computers

- Software 1.0 was explicit rules (writing code). Software 2.0 was learned weights (training neural networks on datasets). Software 3.0 is prompting LLMs that act as programmable interpreters
- Traditional installers are shell scripts targeting specific platforms. OpenClaw's installer is text you paste to an agent, which intelligently adapts to your environment and debugs in the loop
- Menu Gen example: Built a full app to OCR menus and generate food images. Software 3.0 version: Give Gemini the photo, ask it to overlay images using an image generator. The entire app becomes unnecessary
- This isn't just faster programming—it's automating information processing that couldn't be coded before, like generating knowledge bases from unstructured documents

All of my Menu Gen is spurious. It's working in the old paradigm. That app shouldn't exist.

Vibe Coding vs Agentic Engineering

- Vibe coding raises the floor—anyone can now build software by describing intent. Agentic engineering preserves the professional quality ceiling while moving faster
- Agentic engineering is a discipline: coordinating powerful but stochastic agents without sacrificing security, reliability, or maintainability
- The speedup exceeds 10x for skilled practitioners. The gap between mediocre and excellent agentic engineers is widening, not narrowing
- Hiring must evolve: stop giving puzzles. Assign large projects (build a Twitter clone for agents, make it secure, then try to break it with AI red teams) and evaluate execution quality

The Agent-Native Future

- Everything is still designed for humans. Documentation, deployment workflows, configuration UIs—all assume human operators. This must be rewritten
- The ideal: describe Menu Gen to an LLM, and it's deployed on the internet without touching DNS settings, service configurations, or deployment pipelines
- Future infrastructure will be sensors (observing state) and actuators (taking action), with agent-first interfaces and data structures optimized for LLM legibility
- We're moving toward agent representation for people and organizations—your agent talks to my agent to handle logistics, scheduling, and coordination

Why are people still telling me what to do? I don't want to do anything. What is the thing I should copy-paste to my agent?

What Remains Worth Learning

- Intelligence is getting cheap, but understanding remains a human bottleneck. You can't direct agents effectively without comprehending the problem space
- Tools that enhance understanding (LLM knowledge bases, synthetic data generation over fixed datasets, multiple projections of information) become critical
- Deep learning of fundamentals still matters—not API syntax, but core concepts that let you evaluate agent output and make architectural decisions
- The goal isn't to memorize details but to build mental models that let you be an effective director of increasingly autonomous systems

You can outsource your thinking, but you can't outsource your understanding. LLMs don't excel at understanding—you're still uniquely in charge of that.

]